

INF-20325 Estructuras de Datos, 2026-2

DI – UTFSM

Profesores: José-Miguel Cazorla, Andrés Navarro, Jorge Díaz, Ricardo Salas

Pre-Certamen 1

11 de Septiembre de 2026

A1 - Optimizando C++ (30 pts)

El comité de estandarización de C++ se ha encontrado con un problema de eficiencia debido a las nuevas tendencias. Muchos algoritmos basados en listas necesitan una forma eficiente de obtener el 7 más cercano a cada 6 de una lista.

Para esto, quieren incluir a la implementación de lista basada en arreglo un nuevo método para la clase, `int* distancia67()` que retorna un arreglo de largo de la lista original en el que cada posición i contiene:

- Si el elemento en la posición i de la lista es un 6: debe indicar el índice al 7 más cercano de la lista en cualquier dirección. En caso de empate entre dos 7, almacene el índice del 7 anterior al elemento de la lista. Si no hay ningún 7, su valor es -1.
- Si el elemento en la posición i no es un 6, su valor debe ser -1.

Además, dado que este método se usará de forma intensiva, el comité le ha pedido que su complejidad sea $O(N)$, en dónde N es el largo de la lista. Agregue al método los parámetros que estime conveniente.

Restricciones

- $0 \leq N \leq 100000$
- Los valores de la lista son enteros en $[-1000, 1000]$

Entrada

El método no recibe la lista como parámetro, pero previamente se crea la lista de leyendo:

- Primera línea: N , el número de elementos de la lista.
- Segunda línea: N enteros correspondientes a los valores del arreglo.

Salida

Un arreglo con el índice del 7 más cercano para cada 6.

Ejemplos

Ejemplo de entrada	Ejemplo de salida
8 3 6 5 7 6 6 2 7	-1 3 -1 -1 3 3 -1 -1
5 6 6 6 6 7	4 4 4 4 -1
1 6	-1

B1 – Inversión de Anomalías (Sublistas de Pares) (30 pts)

Un sistema de transmisión espacial almacena paquetes de datos numéricos en una **lista doblemente enlazada**. Se ha detectado una anomalía electromagnética en el canal: siempre que se transmite una ráfaga de números pares consecutivos, el orden de dichos números se invierte al llegar a la nave.

Para corregir la señal, se solicita implementar el método interno `void revertirPares()` dentro de la clase `tLista`. Este método debe buscar todas las sublistas contiguas compuestas **exclusivamente por números pares** (de largo ≥ 2) e invertir el orden de sus nodos.

Restricciones

- No se permite crear nuevos nodos (`new`) ni intercambiar los valores `info`.
- Debe usar punteros `tNodo*` internos; **está estrictamente prohibido** usar operaciones de alto nivel como `moveToPos()`, `insert()` o `erase()` dentro del método.
- La complejidad de la solución debe ser estrictamente $O(N)$ en tiempo.
- $1 \leq N \leq 10^5$, $0 \leq \text{datos} \leq 10^6$.

Entrada

La primera línea contiene un entero N , la cantidad de datos en la señal. La segunda línea contiene los N enteros de la señal con las anomalías.

Salida

Los elementos de la lista tras haber revertido internamente las sublistas de números pares, separados por espacios.

Ejemplos

Ejemplo de entrada	Ejemplo de salida
8 1 2 4 6 3 8 10 5	1 6 4 2 3 10 8 5
4 2 4 6 8	8 6 4 2
5 1 3 5 7 9	1 3 5 7 9

C1 – Detección de duplicados

Una universidad mantiene un registro con los identificadores de los estudiantes inscritos en una actividad. Debido a un posible error en el sistema, se desea verificar si existe al menos un estudiante cuyo identificador aparezca más de una vez. Por ejemplo, para el siguiente arreglo de identificadores: [15, 27, 31, 42, 27, 53] el algoritmo debe retornar `true`, ya que el identificador 27 se encuentra repetido.

Se proponen las siguientes tres implementaciones para resolver el problema. Considere que existen n estudiantes registrados. Además, asuma que el algoritmo `sort` tiene una complejidad de $O(n \log n)$. Por último, asuma que el rango de los identificadores de los estudiantes es $[0, 999999]$.

Implementación 1

```
bool existenDuplicados1(vector<int>& ids) {  
  
    int n = ids.size();  
    for (int i = 0; i < n; i++) {  
        for (int j = i + 1; j < n; j++) {  
            if (ids[i] == ids[j]) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

Implementación 2

```
bool existenDuplicados2(vector<int>& ids) {  
  
    int n = ids.size();  
    sort(ids.begin(), ids.end());  
    for (int i = 1; i < n; i++) {  
        if (ids[i] == ids[i - 1]) {  
            return true;  
        }  
    }  
    return false;  
}
```

Implementación 3

```
bool existenDuplicados3(vector<int>& ids) {  
  
    bool vistos[1000000] = {false};  
    int n = ids.size();  
    for (int i = 0; i < n; i++) {  
        if (vistos[ids[i]]) {  
            return true;  
        }  
        vistos[ids[i]] = true;  
    }  
    return false;  
}
```

1. Determine la complejidad temporal en notación Big O de cada una de las tres implementaciones. Justifique brevemente su respuesta.
2. Ordene las tres implementaciones desde la menos eficiente hasta la más eficiente para valores grandes de n .
3. ¿Qué desventaja presenta la Implementación 3 respecto de las otras soluciones?

Formulario

C++

- Tipos de datos primitivos: int, bool, float, char
- Operaciones:

Aritméticos	Comparación	Lógicos	Asignación	Incremento/Decremento
+, -, *, /, %	==, !=, <, <=, >, >=	&&, , !	=, +=, -=, *=, /=	++, --

- Sentencias principales:

```
if (condición){ /* instrucciones si condición es verdadera */ } else { /* instrucciones si condición es falsa */ }
while (condición){/* instrucciones a ejecutar mientras condición sea verdadera */}
for ( asignacion-inicial ; condicion ; incremento ){ /* operación a llevar a cabo en el for */ }
```

- Punteros:

```
int *p; /* puntero de tipo entero, contiene direcciones de memoria de enteros */
int i = 0; /* entero i vale 0 */
p = &i; /* p contiene la dirección de memoria del entero i */
(*p)++; /* se incrementa en 1 el contenido (no la dirección) apuntado por p, ahora vale 1 */
p++; /* se incrementa en 1 la dirección de memoria apuntada por p, ahora ya no apunta a i */
```

- Arreglos

```
int A[100]; /* arreglo de enteros de tamaño 100 */
```

- Estructura

```
struct Persona {int numero; string nombre;};/* se declara estructura llamada Persona que contiene atributos numero y nombre */
Persona p1; /* se crea p1, variable de tipo Persona */
p1.numero = 1; /* se asigna 1 como número del Persona p1 */
p1.nombre = "Juan"; /* se asigna Juan como nombre del Persona p1 */
Persona *p2; /* puntero de tipo Persona */
p2->numero = 4; /* operador -> se usa para asignar en caso de que se tenga un puntero a una estructura */
p2->nombre = "Pablo";
```

- Memoria Dinámica

```
int *A; /* puntero de tipo entero llamado A */
A = new int[100]; /* Se asigna memoria para un arreglo de tamaño 100 cuyo primer elemento es apuntado por A */
Persona *personas; /* puntero de tipo Persona llamado personas */
personas = new Persona[151]; /* Se asigna memoria para 151 personas */
delete[] personas; /* Se libera la memoria utilizada para los 151 Persona */
```

- Paso por referencia

```
float funcion(int i, int& j){
    j+=i; /* la variable j es modificada y se incrementa en i. Este cambio se ve reflejado fuera de la función. */
}
```

- Archivos

```
fstream file;
file.open("archivo.txt", ios::in); /* Se abre en modo lectura ASCII */
file.open("archivo.txt", ios::out); /* Se abre en modo escritura ASCII */
int x;
file >> x; /* leer una variable x */
file << x; /* escribir el valor de una variable x */
file.open("archivo.dat", ios::in | ios::binary); /* Se abre en modo lectura Binario*/
file.open("archivo.dat", ios::out | ios::binary); /* Se abre en modo escritura Binario*/
file.read((char*) &x, bytes_por_leer); /* se lee en binario el valor de la variable x*/
file.write((char*) &x, bytes_por_escribir); /* se escribe en binario el valor de la
    variable x */
file.close()
```

Análisis de Algoritmos

- Regla de la Suma: $f_1 \in O(g), f_2 \in O(h) \Rightarrow f_1 + f_2 \in O(\max\{g, h\})$
- Regla del Producto: $f_1 \in O(g), f_2 \in O(h) \Rightarrow f_1 \cdot f_2 \in O(g \cdot h)$
- Regla del Límite: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = k$
- Si $k \neq 0, k \neq \infty \Rightarrow O(f) = O(g)$
- Si $k = 0 \Rightarrow f \in O(g), g \notin O(f)$
- **Bonus:** Regla de L'Hôpital: Sean f y g funciones continuas en el intervalos $[a, b]$ y $c \in [a, b]$ tal que $f(c) = g(c) = 0$ y $g'(x) \neq 0$ si $x \neq c$, entonces:

$$\lim_{x \rightarrow c} \frac{f(x)}{g(x)} = \lim_{x \rightarrow c} \frac{f'(x)}{g'(x)} = L$$